

ORDER FLOW ENGINE

Execution Plan & Project Delivery Document

How we build it · Who builds each part · When it ships · How we verify it
Claude Code agent pipeline · Phase gates · Risk register · Launch checklist

v1.0 | June 2026 | Living Document — updated each sprint

1. Purpose & Scope

This is the Execution Plan — a living project delivery document separate from the Functional Specification. Where the Functional Specification answers 'What does the software do?', this document answers:

- HOW do we build it — tools, agent pipeline, development workflow
- WHO builds each part — which Claude Code agent is responsible for which deliverable
- WHEN does each piece ship — phase-by-phase timeline with week-level granularity
- HOW do we know it works — acceptance criteria, test gates, and review checkpoints
- WHAT can go wrong — risk register with mitigations
- HOW do we launch — go-to-market checklist and launch sequence

***i** This document references the following specification documents produced earlier in this project. All are considered inputs to this execution plan: Functional Specification v4.0 · External Developer SDK Spec · Multi-Symbol Scaling Spec · Data Feed Integration Spec · Symbol Count Product Spec · Indicator Formulas Doc.*

2. What We Are Building — Executive Summary

For any team member or stakeholder joining mid-project, here is the complete one-page summary of what the Order Flow Engine is:

Component	Description	Delivered By
C++ Core Engine	Symbol-sharded parallel processing engine. Ingests tick data from Kinetick, eSignal, or IQFeed. Computes all 26 order flow indicators (Delta, CVD, Imbalances, Volume Profile, VWAP, Pulse, Turns, Ratio, and more) in real time for 1 to 1000+ symbols simultaneously.	Code Agent (C++)
NinjaTrader 8 Adapter	C# AddOn (thin bridge) that connects Kinetick data into the C++ engine via local TCP. Renders footprint charts, signals, and indicators on NT8 workspaces.	Code Agent (C#)
Data Feed Adapters	Three independent adapters: Kinetick (via NT bridge), IQFeed (direct TCP), eSignal (COM/NT). All normalise ticks to Universal Tick Record format.	Code Agent (C++)
External Developer SDK	REST API + WebSocket streaming exposing all 26 indicators to external developers. OFE-Script declarative signal composition language. Backtesting API. Webhook alerts. Native Plugin SDK (C++, Python, C#).	Code Agent (Go/Rust/C++)
License & Security Engine	Hardware-bound RSA-2048 license token. Symbol count enforcement. Feature gating by tier. Binary obfuscation (Themida/VMProtect). Anti-tamper runtime checks.	Security Agent
Subscription Backend	Paddle/Stripe payment processing. License server (Go). Tier management. Symbol add-on packs. Upgrade/downgrade flows.	Code Agent (Go)
Developer Portal	React SPA for API key management, OFE-Script editor, backtester, alert configuration, plugin uploads, and interactive API documentation.	Frontend Agent
Test Suite	Google Test unit tests for all C++ modules. Historical tape replay harness. Synthetic tick stress generator. API integration tests. NinjaTrader live beta.	Test Agent

3. Claude Code Agent Roster

The project uses a specialised agent pipeline running on Claude Code. Each agent has a defined scope, input set, output set, and handoff protocol. Agents run sequentially within a phase but can run in parallel across independent workstreams.

3.1 Agent Definitions — Complete List

Agent ID	Agent Name	Language / Tool	Primary Responsibility	Inputs	Outputs
AGT-01	Architecture Agent	Claude Code + Markdown	Translate functional spec into full system architecture: component diagrams, interface definitions, data flow, module boundaries	Functional Spec v4.0, all sub-specs	Architecture document, UML diagrams, module dependency map, interface contracts
AGT-02	Interface Agent	Claude Code + C++	Generate ALL C++ header files (.h) — zero implementation. Every class, struct, enum, function signature, and constant.	Architecture document from AGT-01	Complete set of .h header files for all C++ modules. Zero .cpp files.
AGT-03	Core Engine Agent	Claude Code + C++	Implement the C++ core engine: Tick Router, Symbol Worker, Lee-Ready classifier, bar construction, ring buffer, Universal Tick Record.	Approved headers from AGT-02	tick_router.cpp, symbol_worker.cpp, bar_engine.cpp, lee_ready.cpp
AGT-04	Analytics Agent	Claude Code + C++	Implement all 26 indicator calculations per the formula spec: Delta, CVD, Imbalances, Absorption, COT, Trapped Traders, Volume Profile, VWAP, Pulse, Turns, Ratio, Single Prints, Delta Surge, Market Sweep, POC Slingshot.	Approved headers + Core Engine from AGT-03	delta_engine.cpp, imbalance_detector.cpp, volume_profile.cpp, vwap_engine.cpp, signal_detector.cpp, pulse_engine.cpp, turns_engine.cpp
AGT-05	Data Feed Agent	Claude Code + C++/C#	Implement three data feed adapters: IQFeed (direct TCP), Kinetick (NT bridge C# + C++ receiver), eSignal (COM bridge). All output	Approved headers, Data Feed Spec	iqfeed_adapter.cpp, kinetick_nt_adapter.cs, kinetick_cpp_receiver.cpp,

			Universal Tick Records.		esignal_adapter.cpp
AGT-06	NinjaTrader Agent	Claude Code + C#	Implement the NinjaTrader 8 AddOn: footprint chart renderer, indicator overlays, signal alert panels, configuration UI, workspace manager.	Approved C# headers + NT8 SDK docs	OFEAddOn.cs, FootprintChart.cs, SignalPanel.cs, ConfigWindow.cs
AGT-07	API Agent	Claude Code + Go	Implement REST API server, WebSocket server, OFE-Script evaluator, backtesting engine, alert delivery system. OpenAPI 3.1 spec generation.	Approved API spec from SDK doc	api_server.go, websocket_server.go, script_evaluator.rs, backtest_engine.cpp, alert_worker.go
AGT-08	SDK Agent	Claude Code + Python/C#/JS	Generate official client SDK libraries: ofe-python-sdk, OFE.SDK.DotNet, ofe-js-sdk. Includes auth helpers, WebSocket reconnection, type-safe response models.	REST + WebSocket API spec	ofe_sdk/ (Python), OFE.SDK/ (C#), ofe-js-sdk/ (npm)
AGT-09	License Agent	Claude Code + C++/Go	Implement hardware fingerprinting, RSA-2048 license token generation/validation, symbol count enforcement in engine, license server backend, Paddle/Stripe webhook handlers.	Security design from spec	license_engine.cpp, hw_fingerprint.cpp, license_server.go, billing_webhook.go
AGT-10	Test Agent	Claude Code + C++ (gtest)	Generate complete test suite: unit tests for all 26 indicator formulas, synthetic tick generator, historical replay harness, API integration tests, WebSocket load tests.	All .cpp implementations	test_delta.cpp, test_imbalances.cpp, test_vwap.cpp, test_volume_profile.cpp, test_pulse.cpp, test_api.py, test_websocket.js, tick_generator.cpp
AGT-11	Frontend Agent	Claude Code + React/TypeScript	Build the Developer Portal: 10-page React SPA covering API keys, OFE-Script editor, backtester, alerts,	API spec, portal design requirements	portal/ (React app), openapi.yaml, docs/

			plugin uploads, interactive docs, playground.		
AGT-12	Security Agent	Claude Code + Bash/Config	Configure binary protection pipeline: Themida/VMProtect configuration, symbol stripping, PDB removal, anti-debug integration, CI/CD security gates.	Compiled C++ binaries	Protected DLL, build pipeline scripts, security audit report
AGT-13	DevOps Agent	Claude Code + YAML/Bash	Set up infrastructure: CMake build system, GitHub Actions CI/CD, Docker containers, AWS deployment scripts, TimescaleDB + Redis + Kafka setup, monitoring (Prometheus + Grafana).	All source code	CMakeLists.txt, .github/workflows/, docker-compose.yml, terraform/, k8s/
AGT-14	Review Agent	Claude Code (review mode)	Final code audit across all modules: consistency check, gap analysis, dead code detection, API contract verification, security review, formula correctness check against spec.	All generated code	Code review report, gap list, fix tickets

3.2 Agent Dependency Graph

Agents must execute in the order defined by their dependencies. The following shows which agents must complete before the next can start:

Agent	Depends On	Can Run In Parallel With
AGT-01 Architecture	Functional Spec v4.0 (human review complete)	Nothing — must be first
AGT-02 Interface	AGT-01 complete + HUMAN GATE: architecture approved	Nothing — must follow AGT-01
AGT-03 Core Engine	AGT-02 complete + HUMAN GATE: headers approved	Nothing — headers must be locked first
AGT-04 Analytics	AGT-03 complete	AGT-05 (Data Feed) — parallel after Core Engine done
AGT-05 Data Feed	AGT-03 complete	AGT-04 (Analytics) — parallel
AGT-06	AGT-04 + AGT-05 complete	AGT-07 (API) — parallel

NinjaTrader		
AGT-07 API	AGT-04 complete	AGT-06 (NinjaTrader) — parallel
AGT-08 SDK	AGT-07 complete (API stable)	AGT-06 (NinjaTrader) — parallel
AGT-09 License	AGT-03 complete (engine exists)	AGT-04, AGT-05, AGT-06, AGT-07 — all parallel
AGT-10 Test	AGT-04 + AGT-05 complete (code to test)	AGT-06, AGT-07, AGT-08 — parallel
AGT-11 Frontend	AGT-07 complete (API to call)	AGT-06, AGT-08, AGT-09, AGT-10 — parallel
AGT-12 Security	AGT-03 + AGT-04 + AGT-05 compiled and tested	AGT-07, AGT-08, AGT-10, AGT-11 — parallel
AGT-13 DevOps	AGT-03 complete (something to build)	All agents — runs throughout project
AGT-14 Review	ALL agents complete	Nothing — must be last

3.3 Human Review Gates — Non-Negotiable Checkpoints

The following gates require YOU (the product owner) to review and approve before the next agent can proceed. These are the most critical quality checkpoints in the project.

Gate ID	After Agent	What You Review	Time Budget	Why This Gate Exists
GATE -01	AGT-01 Architecture	Full architecture document, component diagram, module boundaries, data flow. Does this match your vision? Any missing components?	2–4 hours	Wrong architecture = weeks of wasted code. Cheapest fix is here.
GATE -02	AGT-02 Interface	ALL C++ header files. Every class, function signature, data type. Does the API between modules make sense? Are the indicator formulas represented correctly in the interfaces?	4–8 hours (CRITICAL)	This is the most important gate. Once code is generated against these headers, interface changes cascade everywhere. Spend maximum time here.
GATE -03	AGT-03 Core Engine	Review core engine code for thread safety, ring buffer correctness, Lee-Ready implementation. Run the unit tests.	2–4 hours	Core engine bugs corrupt every indicator above it.
GATE -04	AGT-04 Analytics	Verify indicator formulas match the formula spec exactly. Check delta calculation, value area algorithm, VWAP standard	4–6 hours	Wrong formula = wrong signals = wrong trades.

		deviation. Compare outputs against worked examples in the spec.		
GATE-05	AGT-10 Test	Review test coverage. Are all 26 formulas tested? Are edge cases covered? Do historical replay results look correct?	2–3 hours	Uncaught formula errors reach live users.
GATE-06	AGT-06 NinjaTrader	Open NinjaTrader 8. Load the AddOn. Does the footprint chart look correct? Do signals appear at the right bars? Does it match Valtos's book examples?	Full trading session	Only you can judge if it looks right vs the source material.
GATE-07	AGT-09 License	Test the license enforcement: try to exceed symbol limit, try to tamper with the DLL, test expiry behavior, test hardware fingerprint.	2–3 hours	License weakness = revenue loss at scale.
GATE-08	AGT-14 Review	Read the complete review report. Prioritise and assign all critical and high issues. Sign off before launch.	3–4 hours	Final quality gate before real users.

4. Phased Execution Plan — Week by Week

The project is divided into 7 phases. Each phase has defined deliverables, responsible agents, acceptance criteria, and a gate before the next phase starts. Timeline assumes one experienced developer directing the Claude Code agent pipeline full-time.

Phase 0 — Foundation & Architecture (Week 1–2)

Item	Detail
Duration	2 weeks
Primary Agent	AGT-01 Architecture Agent
Supporting	AGT-13 DevOps Agent (parallel — sets up repo and build system)
Goal	Complete architecture before a single line of implementation code is written
Ends With	GATE-01 (Architecture review) + GATE-02 (Header review)

Week 1 — Architecture Agent (AGT-01)

Day	Task	Output
Mon	Feed AGT-01: Functional Spec v4.0 + all sub-specs + NinjaTrader addon docs. Prompt: generate full architecture.	Architecture draft v1
Tue	Review architecture draft. Identify gaps. Feed back corrections. Re-run AGT-01.	Architecture draft v2
Wed	AGT-01: Generate UML component diagram, data flow diagram, module dependency map	Diagrams + module list
Thu	AGT-01: Define all inter-module interfaces as text contracts (not yet code)	Interface contracts document
Fri	GATE-01: You review full architecture. Approve or request changes.	Architecture APPROVED

Week 2 — Interface Agent (AGT-02)

Day	Task	Output
Mon	AGT-02: Generate C++ headers for Tick Router, Symbol Worker, Universal Tick Record, ring buffer	tick_router.h, symbol_worker.h, tick_record.h
Tue	AGT-02: Generate headers for all 26 indicators: delta, imbalances, volume profile, VWAP, signals	delta_engine.h, imbalance_detector.h, volume_profile.h, vwap_engine.h, signal_types.h

Wed	AGT-02: Generate headers for data feed adapters (all 3), license engine, API server	adapter_interface.h, license_engine.h, api_server.h
Thu	AGT-02: Generate C# headers for NinjaTrader adapter, OFE-Script types	OFEAdapter.cs (interface only), ScriptTypes.cs
Fri	GATE-02: Full header review — your most important 4–8 hours of the project. Approve all headers.	ALL HEADERS APPROVED

Week 2 Parallel — DevOps Agent (AGT-13)

- Set up GitHub repository with branch protection rules
- Configure CMake build system with all module targets
- Set up GitHub Actions CI: build + test on every push
- Provision AWS infrastructure: EC2 + RDS (TimescaleDB) + ElastiCache (Redis)
- Configure Docker containers for engine, API server, license server

 **GATE: Phase 0 exit gate: All headers approved by you. Repository set up. Build system green. NO implementation code written yet.**

Phase 1 — Core Engine (Week 3–6)

Item	Detail
Duration	4 weeks
Primary Agent	AGT-03 Core Engine Agent
Supporting	AGT-05 Data Feed Agent (starts Week 5), AGT-13 DevOps (CI integration)
Goal	Working tick ingestion pipeline. Ticks flow from any data provider → Symbol Worker → bar construction → delta computation. No signal detection yet.
Ends With	GATE-03 (Core Engine review) + all core unit tests passing

Week 3 — Tick Infrastructure

Agent	Task	Deliverable	Test
AGT-03	Implement Universal Tick Record struct + serialisation	tick_record.cpp	Unit: serialise/deserialise round-trip
AGT-03	Implement SPSC lock-free ring buffer	ring_buffer.h/.cpp	Unit: overflow, underflow, concurrent read/write
AGT-03	Implement Tick Router (MPMC — routes by symbol_id)	tick_router.cpp	Unit: 10 symbols, verify routing correctness
AGT-13	Wire CMake targets, enable	CMakeLists.txt updated	CI: build + asan/tsan

	sanitizers (ASAN/TSAN)		pass
--	------------------------	--	------

Week 4 — Symbol Worker & Lee-Ready

Agent	Task	Deliverable	Test
AGT-03	Implement Symbol Worker: per-symbol thread, state allocation, tick processing pipeline	symbol_worker.cpp	Unit: spin up 10 workers, route 1000 ticks each
AGT-03	Implement Lee-Ready algorithm (Quote Rule + Tick Rule + IQFeed override)	lee_ready.cpp	Unit: all 5 classification cases, known price/bid/ask inputs
AGT-03	Implement bar construction: TIME, RANGE, VOLUME types	bar_engine.cpp	Unit: 1000 ticks → correct bars, OHLCV verified
AGT-03	Implement price level accumulation (bid_vol[], ask_vol[] per bar)	price_level.cpp	Unit: verify bid/ask bucketing against known tick stream

Week 5 — Data Feed Adapters (AGT-05 starts)

Agent	Task	Deliverable	Test
AGT-05	Implement IQFeed adapter: TCP socket, protocol 6.2 parser, SELECT FIELDS, watch/unwatch	iqfeed_adapter.cpp	Integration: connect to IQConnect.exe, subscribe ES, verify ticks arrive
AGT-05	Implement Kinetick bridge: C# NT AddOn sends ticks → local TCP → C++ receiver	kinetick_nt_adapter.cs + kinetick_cpp_receiver.cpp	Integration: NT8 + Kinetick → verify ticks in engine
AGT-05	Implement failover logic: primary → backup on 5s silence	feed_manager.cpp	Unit: simulate IQFeed disconnect, verify Kinetick takes over
AGT-03	Wire Tick Router to receive from all adapters simultaneously	tick_router.cpp (updated)	Integration: 3 providers, 10 symbols each, verify no tick loss

Week 6 — Integration & GATE-03

- Full end-to-end pipeline test: IQFeed → Tick Router → Symbol Workers → Bars
- Run 1-hour live tick replay from IQFeed history port (port 9100)
- Verify bar construction matches NinjaTrader's bar data for same period
- Performance benchmark: 100 symbols × 500 ticks/sec — no dropped ticks, latency < 2ms

 **GATE: GATE-03: You review Core Engine. Tick pipeline working. Lee-Ready classification correct. Bar data matches reference. All unit tests green. Approve before Phase 2.**

Phase 2 — Analytics Engine (Week 7–12)

Item	Detail
Duration	6 weeks
Primary Agents	AGT-04 Analytics Agent (indicators), AGT-10 Test Agent (parallel)
Supporting	AGT-09 License Agent (starts Week 9)
Goal	All 26 indicators computed correctly. Formula outputs match the formula spec. Signals fire at correct bars.
Ends With	GATE-04 (Formula verification) + GATE-05 (Test coverage review)

Week 7 — Delta Engine

Agent	Indicator	Formula Reference	Acceptance Criteria
AGT-04	Bar Delta, Max/Min Delta, Delta%	Spec §10 formulas 2.1–2.3	Output matches worked example in spec. Bar Delta = ask_vol_sum - bid_vol_sum verified.
AGT-04	Cumulative Delta (CVD)	Spec §10 formula 2.4	CVD resets at session open. Session total matches sum of all bar deltas.
AGT-04	Delta Divergence (standard + bar-level)	Spec §10 formula 2.5	Fire on known divergence patterns from Valtos book chapter examples.
AGT-10	Unit tests: all delta formulas	gtest suite	100% pass on all known-input/known-output test cases.

Week 8 — Imbalance & Absorption

Agent	Indicator	Formula Reference	Acceptance Criteria
AGT-04	Single Buying/Selling Imbalance	Spec §11 formula 3.1	Diagonal comparison verified. Ratio 3.0 = flags correctly. No false positives on balanced levels.
AGT-04	Stacked Imbalance (zone creation, persistence, zone_id)	Spec §11 formula 3.2	3+ consecutive imbalances → zone created. Zone persists across bars. Price-through resolves zone.
AGT-04	2nd Slot, Inverse, Reload, Multiple Imbalances	Spec §4.3.3	All 5 imbalance subtypes fire on matching conditions.
AGT-04	Absorption detection	Spec §11 formula 3.4	High volume + near-zero delta flags correctly. Low volume + near-zero delta

			does NOT flag.
AGT-10	Unit tests: all imbalance formulas	gtest suite	Tests for ratio boundaries, stacking threshold, absorption thresholds.

Week 9 — Volume Profile

Agent	Indicator	Formula Reference	Acceptance Criteria
AGT-04	Profile construction (all types: session, daily, weekly, flexible, composite)	Spec §12 formula 4.1	Profile histogram matches tick-level volume at each price bucket.
AGT-04	POC calculation	Spec §12 formula 4.2	POC = highest-volume price level. Tie-break: lower price.
AGT-04	Value Area (two-level expansion algorithm)	Spec §12 formula 4.3	Value area contains $\geq 70\%$ of session volume. Matches NinjaTrader VP output for same session.
AGT-04	HVN, LVN detection + Shape classification (D/P/b/Thin)	Spec §12 formulas 4.4–4.6	HVN/LVN thresholds verified. Shape matches visual observation on test session.
AGT-09	License engine: hardware fingerprint, RSA token generation	Security spec	Fingerprint unique per machine. Token signed, verified, expiry enforced.

Week 10 — VWAP & COT

Agent	Indicator	Formula Reference	Acceptance Criteria
AGT-04	Daily, Weekly, Yearly VWAP (incremental update)	Spec §13 formula 5.1	VWAP matches Thinkorswim VWAP output for same ES session to 4 decimal places.
AGT-04	Anchored VWAP (all 8 anchor types)	Spec §13 formula 5.2	Anchored VWAP resets correctly at user-specified timestamp. Matches TradingView anchored VWAP.
AGT-04	VWAP standard deviation bands (volume-weighted)	Spec §13 formula 5.3	$\pm 1\sigma$ and $\pm 2\sigma$ bands computed correctly. Verified against known ES session reference data.
AGT-04	COT (intrabar) — highest volume price level per bar	Spec §14 formula 6.1	COT price = $\text{argmax}(\text{total_vol})$ within bar. Position ratio computed and categorised correctly.

Week 11 — Pulse, Turns, Ratio, Single Prints

Agent	Indicator	Formula	Acceptance Criteria
-------	-----------	---------	---------------------

		Reference	
AGT-04	Orderflows Pulse (7-variable composite, direction detection)	Spec §15 formulas 7.1–7.2	Composite score 0–100. Score ≥ 70 fires signal. Variable scores individually inspectable via API.
AGT-04	Orderflows Turns (6-variable binary score)	Spec §16 formula 8.1	Score 0–6. Score ≥ 3 = MODERATE, ≥ 5 = STRONG. Direction determined by V4 candle.
AGT-04	Orderflows Ratio (Top Heavy / Bottom Heavy)	Spec §17 formula 9.1	Fires at bar extremes only. Max 1 signal per bar. Ratio threshold configurable.
AGT-04	Single Prints (Last Buyer/Seller, exhaustion score, magnet flag)	Spec §18 formula 10.1	Fires on ≤ 2 contracts at bar extreme. Magnet stays active until price revisits.

Week 12 — Remaining Signals + GATE-04 + GATE-05

Agent	Task	Output
AGT-04	Delta Surge, Market Sweep, Trapped Traders, POC Slingshot	delta_surge.cpp, market_sweep.cpp, trapped_traders.cpp, poc_slingshot.cpp
AGT-04	All remaining Trader 7 signals (Accumulation/Distribution, POC Wave, Resting Liquidity, etc.)	trader7_signals.cpp
AGT-10	Full formula verification test suite — every formula against spec worked examples	test_all_formulas.cpp — must be 100% green
AGT-10	Historical tape replay: 5 full ES sessions from IQFeed — verify all signals match expected output	replay_test_results.csv
You	GATE-04: Verify formula outputs against source books. Compare Pulse/Turns/Imbalance signals on NinjaTrader.	Formula correctness APPROVED
You	GATE-05: Review test coverage report. All critical paths covered?	Test coverage APPROVED

 **GATE: Phase 2 exit gate: All 26 indicators implemented and formula-verified. Test suite 100% green. GATE-04 and GATE-05 approved by you. Signal outputs validated against source books.**

Phase 3 — Platform Integration (Week 13–18)

Item	Detail
Duration	6 weeks

Primary Agents	AGT-06 NinjaTrader Agent, AGT-07 API Agent, AGT-08 SDK Agent
Supporting	AGT-10 Test Agent (API tests), AGT-11 Frontend Agent (starts Week 16)
Goal	NinjaTrader 8 fully integrated and usable. External API live and documented. Python SDK working.
Ends With	GATE-06 (NinjaTrader live review) + API integration test suite green

Week 13–14 — NinjaTrader AddOn (AGT-06)

Week	Task	Deliverable	Acceptance
13	Implement footprint chart renderer: bid/ask volume cells, imbalance colouring, delta display, COT box	FootprintChart.cs	Chart renders correctly on NT8 with live Kinetick data
13	Implement signal overlay panel: Pulse, Turns, Ratio, Single Print annotations on chart	SignalOverlay.cs	Signals appear at correct bars matching engine output
14	Implement Volume Profile histogram overlay, VWAP lines + deviation bands	VolumeProfileOverlay.cs, VWAPOverlay.cs	VP and VWAP match reference implementation (Dale's examples)
14	Implement configuration UI: all 27 config parameters exposed as NT property grid	ConfigWindow.cs	All config changes propagate to engine in < 1 second (hot reload)

Week 15–16 — REST + WebSocket API (AGT-07)

Week	Task	Deliverable	Acceptance
15	Implement REST API (Go): all /v1/ endpoints — footprint, delta, imbalances, profile, vwap, signals, quota	api_server.go	All endpoints return correct data. OpenAPI 3.1 spec auto-generated.
15	Implement WebSocket server: all topic subscriptions, fan-out, watchlist streams, heartbeat	websocket_server.go	1000 concurrent connections. P99 latency < 20ms. No dropped messages.
16	Implement OFE-Script engine (Rust): condition parser, evaluator, backtest integration	script_evaluator.rs	5 test scripts execute correctly. Backtest results match manual calculation.
16	Implement alert delivery: webhook (HMAC-signed), Telegram, email, Discord	alert_worker.go	Webhook delivered < 500ms. HMAC signature verifiable.

Week 16–17 — SDK Libraries (AGT-08)

Week	SDK	Key Features	Acceptance
------	-----	--------------	------------

16	Python SDK (of-python-sdk)	OFEClient, WatchlistStream, auto-reconnect, type models, auth helper	pip install works. 5-line script subscribes to ES pulse stream successfully.
17	C# SDK (OFE.SDK.DotNet)	OFEClient, typed WebSocket events, NuGet package	Install-Package works. C# demo script connects and receives signals.
17	JavaScript SDK (of-js-sdk)	npm package, browser + Node.js, Promise-based API, EventEmitter	npm install works. Browser demo page shows live delta for 5 symbols.

Week 17–18 — Developer Portal (AGT-11)

Week	Task	Deliverable
17	API Key management, watchlist manager, quota display pages	portal/src/pages/keys/, watchlist/, quota/
17	OFE-Script editor with syntax highlighting and real-time validation	portal/src/pages/scripts/
18	Backtester UI: run config, equity curve chart, trade list, statistics panel	portal/src/pages/backtest/
18	Interactive API documentation (Stoplight Elements), SDK download page, playground	portal/src/pages/docs/, playground/

 **GATE: GATE-06: You open NinjaTrader 8. Load a live ES chart with footprint. Run for a full trading session. Signal quality matches expectations from Valtos/Dale books. Approve before Phase 4.**

Phase 4 — Security & Licensing (Week 19–22)

Item	Detail
Duration	4 weeks
Primary Agents	AGT-09 License Agent, AGT-12 Security Agent
Supporting	AGT-13 DevOps (deployment pipeline updates)
Goal	Hardware-bound licensing enforced. Binaries obfuscated and protected. Subscription backend live. All tier limits enforced.
Ends With	GATE-07 (License enforcement test)

Week 19–20 — License Server & Enforcement (AGT-09)

Week	Task	Deliverable
19	License server backend (Go): key generation, HW fingerprint validation, RSA	license_server/ (Go service)

	signing, Redis token cache	
19	Paddle webhook handlers: payment_succeeded → activate, payment_failed → grace, chargeback → blacklist	billing_webhook.go
20	Symbol count enforcement in C++ engine (GATE-LAYER-1): engine refuses Symbol Worker allocation above license limit	license_engine.cpp integrated into symbol_worker.cpp
20	Feature flag enforcement: all tier-gated features check license token before activation	feature_flags.cpp

Week 21–22 — Binary Protection (AGT-12)

Week	Task	Tool	Deliverable
21	Strip all debug symbols from release build	MSVC /OPT:REF + strip	Release DLL with no PDB, no symbols
21	Configure Themida protection on DLL: code virtualisation for license check section	Themida 3.x	Protected DLL — passes tamper test
21	Configure VMProtect on critical algorithm sections	VMProtect 3.x	Core formula code virtualised
22	Anti-debug integration: IsDebuggerPresent + NtQueryInformationProcess checks	C++ inline	Engine exits cleanly on debugger attach
22	Binary integrity check: hash .text section at startup, verify matches embedded hash	C++ startup code	Patched DLL detected and rejected
22	CI/CD security gate: automated check that release builds are protected before publish	GitHub Actions step	Build fails if protection not applied

 **GATE: GATE-07: License enforcement test. (1) Try to activate more symbols than tier allows — must be blocked. (2) Copy DLL to second machine — must reject. (3) Modify DLL bytes — must detect and exit. (4) Attach debugger — must detect and exit. All 4 must pass.**

Phase 5 — Testing & Beta (Week 23–28)

Item	Detail
Duration	6 weeks

Primary Agent	AGT-10 Test Agent + You (beta coordinator)
Supporting	AGT-14 Review Agent (Week 27–28), all prior agents (fixing issues)
Goal	All bugs found and fixed. 10 beta users running the software live on NinjaTrader. GATE-08 approved. Ready to ship.
Ends With	GATE-08 (Final review sign-off)

Week 23 — Synthetic Stress Testing (AGT-10)

Test	Configuration	Pass Criteria
Tick storm	500,000 ticks/sec for 60 seconds, 100 symbols	Zero tick loss. No ring buffer overflow. Delta/CVD correct after storm.
Out-of-order timestamps	0.1% of ticks delivered 50ms late	Sequence correction handles all late ticks. No delta corruption.
Symbol churn	Add/remove 10 symbols per second for 5 minutes	No memory leak. No stale Symbol Workers. Quota enforced throughout.
Feed failover	Disconnect IQFeed mid-session	Kinetick takes over within 5 seconds. Signal stream resumes. Gap noted.
API load	1000 WebSocket connections × 100 subscriptions	P99 latency < 50ms. Zero dropped messages. Memory stable.

Week 24–25 — Historical Tape Replay Testing

Session	Instrument	Expected Signals	Pass Criteria
FOMC day (high vol)	ES (5-min)	Multiple stacked imbalances, Delta surges, CVD divergences	All signals fire at bars matching Valtos book examples. No false positives.
Trend day	NQ (4-range)	Pulse LONG sustained, POC slingshot	Pulse fires at session open breakout. Score ≥ 75 confirmed.
Choppy/range day	CL (5-min)	VWAP rotation signals, Turns at extremes	Turns fire at session high/low. VWAP bands respected.
Earnings release	AAPL (1-min stocks)	Delta surge, market sweep, volume profile shift	Delta surge fires within 2 bars of announcement. Market sweep detected.
5 full sessions	ES	All indicator values	Delta/CVD/VWAP match NinjaTrader reference for same periods to 4 d.p.

Week 26 — Beta Launch (10 Users)

- Recruit 10 beta users: 5 NinjaTrader futures traders, 3 stock day traders, 2 developer API users
- Each receives: beta license (Professional tier, 30-day), setup guide, feedback form, Telegram support group
- Monitor: signal quality feedback, latency reports, bug reports, NinjaTrader compatibility issues
- Daily check-in with beta users for first 5 days. Weekly thereafter.
- All reported bugs triaged within 24 hours. Critical bugs fixed within 48 hours.

Week 27–28 — Final Review (AGT-14) + Bug Fix Sprint

Week	Task	Output
27	AGT-14 Review Agent: full code audit across all modules	Review report: critical/high/medium/low issues
27	Fix all critical issues from beta + review	Zero critical open issues
28	AGT-14: re-review fixed items only	Confirmation all critical issues resolved
28	GATE-08: You review full review report and beta feedback summary	Final sign-off

 **GATE: GATE-08: Final sign-off. Zero critical bugs open. Beta user NPS ≥ 8/10 average. All 8 gates previously passed. Approve to proceed to launch.**

Phase 6 — Launch & Scale (Week 29–52)

Item	Detail
Duration	24 weeks (ongoing)
Responsible	You + Support Agent (AGT-15, new) + Community
Goal	NinjaTrader v1.0 public launch. First 100 paying subscribers. Sierra Chart adapter. Web dashboard.
Key Milestones	Week 30: Public launch. Week 36: Sierra Chart adapter. Week 44: Web dashboard beta. Week 52: TradingView bridge.

Platform Expansion Roadmap

Platform	Agent	Timeline	Integration Method
NinjaTrader 8 (v1.0)	AGT-06 (complete)	Week 30 — PUBLIC LAUNCH	C# P/Invoke → C++ DLL
Sierra Chart	AGT-06 (new scope)	Week 36–40	Native C++ ACSIL study
Web Dashboard	AGT-11 (extended)	Week 44–48	React + WebSocket API

TradingView	AGT-07 (extended)	Week 50–54	WebSocket bridge to Pine Script
Bookmap	AGT-06 (new scope)	Month 9–10	JNI Java bridge → C++ engine
Trading Technologies (TT)	AGT-06 (new scope)	Month 10–12	TT SDK adapter
MT5 / cTrader	AGT-06 (new scope)	Month 12–15	MQL5/cAlgo bridge

5. Complete Timeline — One-Page Summary

Week	Phase	Primary Agents	Deliverable	Gate
1	P0 Foundation	AGT-01, AGT-13	Architecture doc, component diagrams, repo setup	GATE-01
2	P0 Foundation	AGT-02, AGT-13	ALL C++ header files, build system	GATE-02
3	P1 Core Engine	AGT-03, AGT-13	Ring buffer, Tick Router, Universal Tick Record	—
4	P1 Core Engine	AGT-03	Symbol Worker, Lee-Ready, bar construction	—
5	P1 Core Engine	AGT-03, AGT-05	Data feed adapters: IQFeed + Kinetick	—
6	P1 Core Engine	AGT-03, AGT-05	Integration + benchmark. End-to-end tick flow	GATE-03
7	P2 Analytics	AGT-04, AGT-10	Delta engine (all 5 formulas)	—
8	P2 Analytics	AGT-04, AGT-10	Imbalances + Absorption (all types)	—
9	P2 Analytics	AGT-04, AGT-09	Volume Profile + License engine start	—
10	P2 Analytics	AGT-04	VWAP + COT	—
11	P2 Analytics	AGT-04	Pulse, Turns, Ratio, Single Prints	—
12	P2 Analytics	AGT-04, AGT-10	Remaining signals + full formula test suite	GATE-04, GATE-05
13–14	P3 Platform	AGT-06	NinjaTrader 8 AddOn complete	—
15–16	P3 Platform	AGT-07	REST API + WebSocket server	—
16–17	P3 Platform	AGT-08	Python, C#, JS SDKs	—
17–18	P3 Platform	AGT-11	Developer Portal (React SPA)	GATE-06
19–20	P4 Security	AGT-09	License server + symbol count enforcement	—
21–22	P4 Security	AGT-12	Binary protection (Themida/VMProtect)	GATE-07
23	P5 Testing	AGT-10	Synthetic stress tests	—
24–25	P5 Testing	AGT-10	Historical tape replay (5 sessions)	—
26	P5 Testing	You + beta users	10-user beta launch	—
27–28	P5 Testing	AGT-14 + all agents	Final code review + bug fix sprint	GATE-08
29	P6 Launch	You + AGT-13	Production deployment, Paddle billing live	—
30	P6 Launch	You	PUBLIC LAUNCH — NinjaTrader v1.0	

36–40	P6 Scale	AGT-06	Sierra Chart adapter	—
44–48	P6 Scale	AGT-11	Web Dashboard beta	—
50–54	P6 Scale	AGT-07	TradingView WebSocket bridge	—

6. Delivery Method — How Each Agent Executes

6.1 Claude Code Session Workflow

Each agent runs as a Claude Code session. The following is the standard workflow for every agent:

Step	Action	Your Role
1 . P r i m e t h e a g e n t	Open Claude Code. Paste agent prompt: 'You are AGT-04 Analytics Agent. Read the following spec sections: [paste relevant sections]. Generate the implementation files.'	Paste spec sections and any prior agent outputs needed
2 . F e e d i n p u t s	Provide approved header files, architecture doc, prior agent outputs, and any relevant formula spec sections	Copy files into Claude Code context
3 . G e n e r a t e	Agent generates implementation files one module at a time	Monitor output. Intervene if agent goes off-track.
4 . R e v i e	Review generated code against spec. Check formula correctness, variable names match headers.	Your active review — do not skip

w		
5 . It e r a t e	If issues found: paste the problem back to agent with specific feedback. 'Line 142: absorption ratio is wrong, should use total_vol not ask_vol only'	Specific feedback — not vague
6 . T e s t	Run gtest suite for this module. If failures: feed error output back to agent for fix.	Run tests yourself or have AGT-10 generate them first
7 . C o m m i t	Commit approved code to repository. Tag with agent ID and version.	git commit + push
8 . H a n d o f f	Pass approved output files to next agent's input set	Document handoff in project log

6.2 Parallel Agent Management

From Phase 2 onwards, multiple agents run simultaneously. Manage them with separate Claude Code sessions:

- Session A (AGT-04 Analytics): working on indicator implementation
- Session B (AGT-10 Test): generating tests for the formulas Session A just produced
- Session C (AGT-09 License): implementing license server (independent of analytics)
- Session D (AGT-13 DevOps): keeping CI/CD green as new code arrives

***i** Parallel sessions work because agents operate on different files. AGT-04 writes .cpp implementation files. AGT-10 writes _test.cpp files. AGT-09 writes Go files. They don't conflict. Merge all to main daily.*

6.3 Prompt Templates for Each Agent

AGT-01 Architecture Agent Prompt

'You are the Architecture Agent for the Order Flow Engine project. Your task is to produce a complete system architecture document. Here are your inputs: [paste Functional Spec v4.0 key sections]. Produce: (1) Component diagram as ASCII art. (2) Module list with single-sentence responsibility. (3) Data flow from tick receipt to signal emission. (4) Inter-module interface contracts as bullet points. (5) Technology stack per layer. Do not write any code. Output documentation only.'

AGT-02 Interface Agent Prompt

'You are the Interface Agent for the Order Flow Engine. Your task is to generate ALL C++ header files (.h) for the system. Here is the approved architecture: [paste AGT-01 output]. Here are the indicator formulas you must represent in the interfaces: [paste formula spec §9-23]. Rules: (1) ONLY headers — no .cpp files. (2) Every class must have a comment explaining its role. (3) Every function must have a comment explaining its parameters and return value. (4) No implementation — only declarations. Generate headers one module at a time.'

AGT-03 Core Engine Prompt

'You are the Core Engine Agent. Implement the C++ files for the tick processing pipeline. Here are the approved headers: [paste AGT-02 headers]. Here is the architecture: [paste AGT-01 doc]. Rules: (1) Implement exactly to the header — do not change any interface. (2) Use lock-free SPSC ring buffer for Symbol Worker queues. (3) Lee-Ready algorithm must implement both Quote Rule and Tick Rule. (4) Write gtest unit tests alongside each implementation file. Start with tick_record.cpp and ring_buffer.cpp.'

AGT-04 Analytics Prompt (example for Delta)

'You are the Analytics Agent. Implement delta_engine.cpp against these approved headers: [paste delta headers]. The exact formulas are: [paste §10 from Indicator Formulas doc]. Rules: (1) $\text{BarDelta} = \text{SUM}(\text{ask_vol}) - \text{SUM}(\text{bid_vol})$ across all price levels. (2) CVD resets to 0 at session_open_ts. (3) Delta divergence detects both standard (CVD vs price) and bar-level (candle vs delta) patterns. (4) Every formula must have a corresponding gtest test with known-input/known-output. Implement delta_engine.cpp now.'

7. Testing Strategy — Complete Test Plan

Test Type	Agent	When	Tools	Pass Criteria
Unit tests — formulas	AGT-10	After each indicator implementation (Weeks 7–12)	Google Test (gtest)	100% pass on known-input/known-output for all 26 formulas
Unit tests — infrastructure	AGT-10	After each core engine component (Weeks 3–6)	gtest + AddressSanitizer	Ring buffer, Lee-Ready, bar construction — all edge cases green
Formula verification	You (GATE-04)	Week 12	Manual: compare to spec worked examples + NinjaTrader	Delta matches NT to 4 d.p. Value Area $\geq$ 70%. VWAP matches TOS.
Synthetic stress test	AGT-10	Week 23	Custom tick generator (C++)	500k ticks/sec, 100 symbols, 0 dropped ticks, latency < 2ms
Historical tape replay	AGT-10	Weeks 24–25	IQFeed port 9100 + replay harness	Signal output matches expected signals from known sessions
API integration tests	AGT-10	Week 16	pytest + websocket-client (Python)	All REST endpoints return correct data. WS delivers signals within 20ms.
WebSocket load test	AGT-10	Week 16	k6 or Artillery (JS)	1000 concurrent connections. P99 latency < 50ms.
NinjaTrader integration	You (GATE-06)	Week 18	NT8 with live data	Full session review. Signals fire at correct bars.
License enforcement	You (GATE-07)	Week 22	Manual tamper tests	All 4 tamper scenarios correctly rejected
Beta user feedback	10 beta users	Weeks 26–28	Feedback form + Telegram	NPS $\geq$ 8/10. Zero critical unresolved bugs at launch.
Final code review	AGT-14	Week 27–28	Claude Code review mode	Zero critical issues open. All high issues resolved or accepted.

8. Risk Register

Risk ID	Risk	Probability	Impact	Mitigation	Owner
R-01	Header interface approved but fundamental flaw found in Week 5+ — requires breaking change	Low (if GATE-02 is thorough)	Very High	Enforce GATE-02 as the most time-intensive review. Involve a second reviewer if possible.	You at GATE-02
R-02	Indicator formula produces different output from NinjaTrader reference	Medium	High	Run formula comparison test against NT immediately after each indicator is implemented. Fix before moving on.	AGT-10 + You
R-03	IQFeed TCP protocol change or documentation gap	Low	Medium	Reference IQFeed official API docs directly. Build protocol version check at adapter startup.	AGT-05
R-04	Kinetick / NinjaTrader version compatibility break (NT acquired by Kraken May 2025)	Medium	High	Test against NT8 v23.x (current stable). Pin NT version in development. Monitor NT release notes.	AGT-06
R-05	Themida/VMProtect licensing cost or integration complexity	Medium	Medium	Budget \$500–\$1000 for Themida license. Allow 2 full weeks for protection configuration (Week 21–22).	AGT-12
R-06	Claude Code context window overflow on large code files	Medium	Medium	Split large modules into smaller files. Never generate more than 500 lines per session. Use file references.	All agents
R-07	Beta user finds fundamental signal quality issue (wrong Pulse formula)	Low-Medium	High	Beta in Week 26 (not Week 30). 4 weeks to fix before public launch.	You + AGT-04
R-08	AWS infrastructure cost exceeds budget	Low	Medium	Start with smallest viable instance (8-core). Scale only when paying users justify cost.	AGT-13
R-09	Paddle or Stripe compliance	Low	Medium	Use Paddle —	AGT-09

	issue (tax, VAT)			handles VAT automatically. Verify compliance in Week 19 before billing goes live.	
R-10	Developer Portal takes longer than 2 weeks (complex React UI)	Medium	Low	Portal is NOT on the critical path for v1.0 launch. NinjaTrader users don't need it. Delay portal to post-launch if needed.	AGT-11

9. Launch Checklist — NinjaTrader v1.0

Every item below must be checked before the public launch announcement. Track in a shared project management tool (Linear, Jira, or Notion).

9.1 Technical Readiness

#	Item	Owner	Status
T-01	All 8 phase gates passed and documented	You	—
T-02	Zero critical bugs open in issue tracker	All agents	—
T-03	Zero high bugs open (or accepted with known workaround)	All agents	—
T-04	Full test suite green (unit + integration + replay)	AGT-10	—
T-05	NinjaTrader 8 AddOn signed with code signing certificate	AGT-12	—
T-06	DLL protected with Themida and passes tamper tests	AGT-12	—
T-07	License server deployed to production AWS (not dev)	AGT-09	—
T-08	Paddle billing configured: all 6 tiers + add-on packs live	AGT-09	—
T-09	SSL certificate valid on all production endpoints	AGT-13	—
T-10	Monitoring live: Prometheus + Grafana dashboard active	AGT-13	—
T-11	Backup and restore tested: DB backup + engine state recovery	AGT-13	—
T-12	IQFeed production credentials (not test) configured	AGT-05	—

9.2 Product Readiness

#	Item	Owner	Status
P-01	NinjaTrader AddOn installation guide published	You	—
P-02	Video tutorial: install + first footprint chart setup (< 10 minutes)	You	—

P-03	FAQ: Kinetick + IQFeed setup questions answered	You	—
P-04	Refund policy and terms of service published on website	You	—
P-05	Pricing page live with all 6 tiers clearly described	You	—
P-06	Beta user testimonials collected and approved for publishing	You + beta	—
P-07	Support email / Telegram group configured for launch traffic	You	—
P-08	orderflowengine.com domain + SSL configured	AGT-13	—

9.3 Launch Sequence (Day of Launch)

Time	Action	Owner
T-2 days	Final smoke test: buy a Starter subscription yourself, install on a fresh NT8 machine, verify full flow	You
T-1 day	Confirm Paddle billing test transactions are removed. Production keys live.	AGT-09
T-0, 06:00 ET	Deploy production build to AWS. Restart all services. Verify health checks green.	AGT-13
T-0, 08:00 ET	Verify Kinetick + IQFeed connections live. ES market opens at 09:30 ET.	You
T-0, 09:30 ET	Monitor first live session: verify ticks, signals, billing. Stay at keyboard.	You
T-0, 10:00 ET	Publish launch announcement: NinjaTrader forums, TraderLion, TradingView community, X/Twitter	You
T-0, 12:00 ET	Check first hour: support tickets, billing issues, error logs	You
T+1 day	Review first 24-hour metrics: signups, conversions, error rate, support volume	You
T+7 days	First week retrospective: what needs fixing before marketing spend increases	You

10. Success Metrics — How We Know It's Working

Metric	Target — Month 1	Target — Month 3	Target — Month 6	How Measured
Paying subscribers	50	200	500	Paddle dashboard
Monthly Recurring Revenue (MRR)	\$2,500	\$12,000	\$35,000	Paddle dashboard
Average subscription tier	Starter/Trader mix	Trader/Professional mix	Professional/Elite mix	Revenue ÷ subscribers
Signal latency (P99)	< 10ms	< 5ms	< 5ms	Prometheus / Grafana
Uptime	99.5%	99.9%	99.9%	AWS + Grafana
Support ticket volume	< 20/week	< 30/week	< 50/week	Helpdesk
Net Promoter Score	≥ 7	≥ 8	≥ 8.5	Monthly email survey
Bug report rate (critical)	0/week	0/week	0/week	Issue tracker
NinjaTrader forum mentions	5+	20+	50+	Manual monitoring
Developer API users	2	10	30	Portal analytics

END OF EXECUTION PLAN — v1.0 | Living Document — Update Each Sprint

14 agents · 6 phases · 28 weeks to launch · 8 human gates · Risk register · Launch checklist · Success metrics